

Dinajpur Railway Station

A 2D Animated Scene in C++ with OpenGL and GLUT

Project Defence Guide — how every part of the code works



Adnan Sami Chowdhury · ID 23-53009-3

B.Sc. in Electrical and Electronic Engineering, AIUB

Computer Graphics · main2_reduced.cpp · 3,569 lines

11 September 2026

1. What this program is

The program opens a window and draws a railway station. Nothing in the window is a photograph or an image file. Every roof, wheel, tree and person is a set of corner points that the program lists out and asks the graphics card to fill in with colour. The picture is rebuilt from scratch about thirty-three times every second, and because a few numbers change a little bit between one rebuild and the next, the eye sees movement.

The scene has three states. On its own it swings between day and night. Pressing the R key starts the rain and freezes the clock where it stands; pressing R again stops the rain and lets the clock run on.

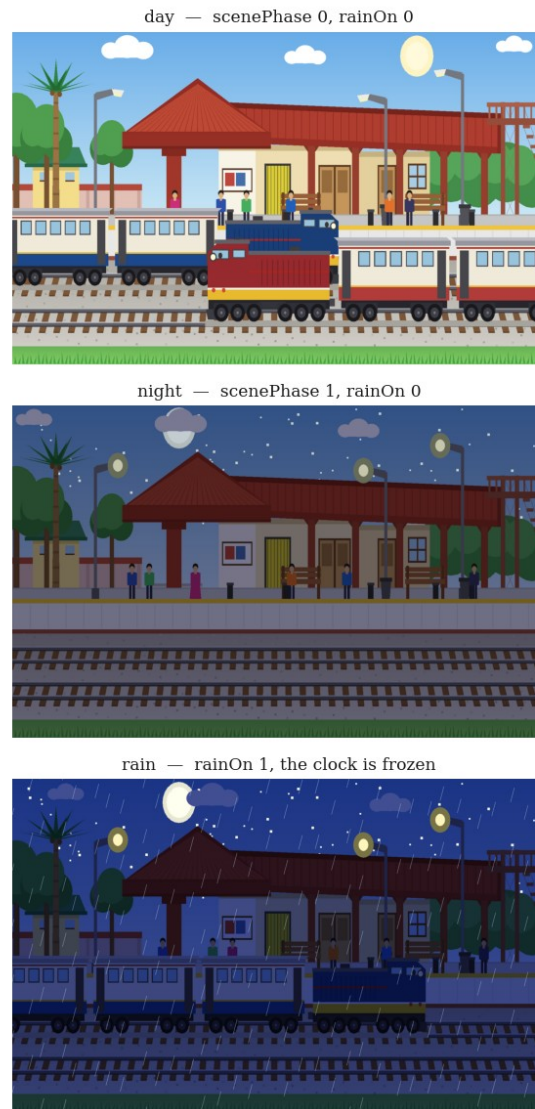


Figure 9 — the three states the scene can be in.

There is no physics, no game engine and no 3D. It is a flat drawing, in one file, using only what the course covered.

The four things you should be able to say in one sentence each

Question	Answer
What draws the picture?	display(), one long list of shapes drawn back to front.
What makes it move?	update(), called every 30 milliseconds, which changes numbers.

Question	Answer
What reacts to the keys?	SpecialInput() for the arrow keys, NormalInput() for R.
What plays the sound?	PlaySound(), a Windows function, started when a flag is raised.

2. The tools, in plain words

Name	What it actually is	What it does for us
C++	the programming language	holds the variables, the if statements and the loops
OpenGL	a drawing library built into the graphics driver	takes lists of points and fills them with colour
GLUT	a helper library on top of OpenGL (we use freeglut)	opens the window, hands us the keyboard, runs the timer
windows.h	a Windows system header	gives us PlaySound for the .wav files

Immediate mode is the phrase for the style of OpenGL used here. It means we hand OpenGL the points one at a time, every single frame, and OpenGL forgets them as soon as the frame is finished. Nothing is stored on the graphics card between frames. It is the simplest way to write OpenGL and the way the course teaches it.

3. The shape of the file

The whole program is one file with eleven functions and one main. Read it in this order, small ones first: the four drawing helpers are only a few lines each, and once they are clear, the long display() function is just a list of calls to them.

Function	Lines	Size	What it does
initGL()	9 – 12	4	sets the colour the window is wiped to
update(int value)	55 – 213	159	the animation: changes every moving number
SpecialInput(...)	215 – 238	24	the arrow keys
NormalInput(...)	241 – 252	12	the R key
drawCircle(...)	256 – 290	35	one round shape, used 92 times
drawClouds(float dx)	293 – 338	46	the three clouds, drawn twice for wrapping
drawCoach(float dx, ...)	343 – 508	166	one passenger coach
drawTrain(...)	513 – 800	288	three coaches plus the engine
drawPassenger(float x, ...)	804 – 848	45	one person on the platform
drawBench(float x)	851 – 894	44	one bench
display()	897 – 3536	2640	draws the whole scene, back to front
main(...)	3540 – 3569	30	sets everything up and starts GLUT

4. How the program starts, and how it keeps running

This is the part most people get wrong in a viva, so it is worth being precise. After `main()` finishes its setup it calls `glutMainLoop()`, and that function never returns. From that moment on, your code is not in charge — GLUT is. GLUT calls your functions when something happens.

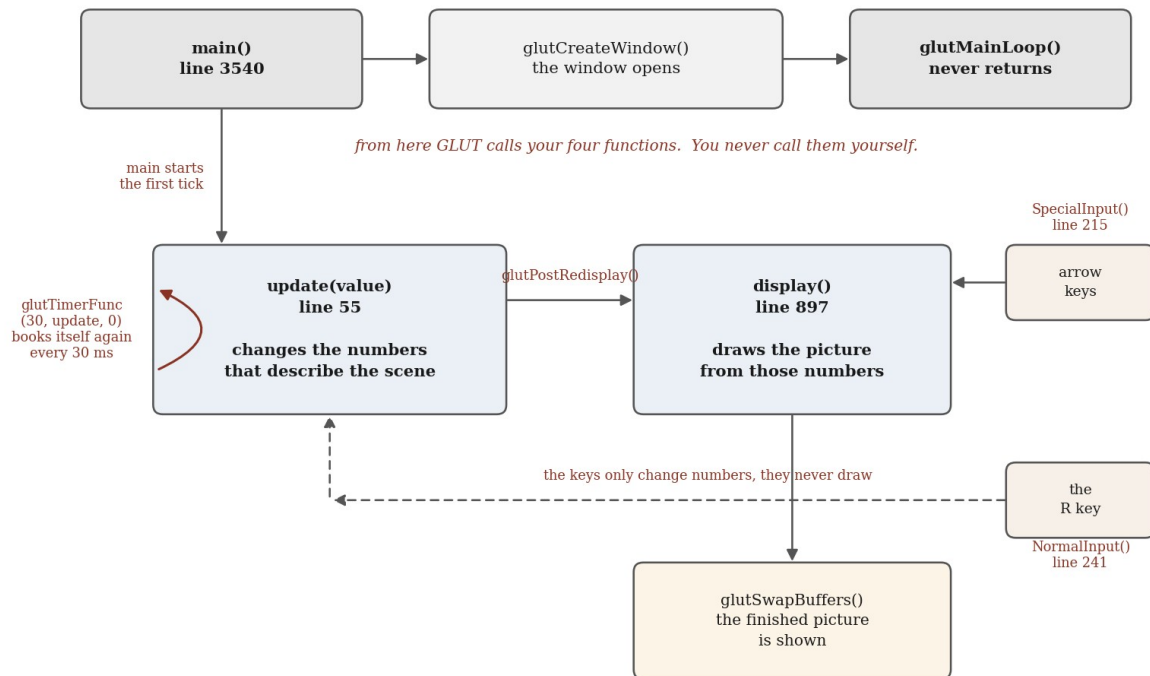


Figure 1 — how the program runs. `update()` and `display()` never call each other directly: `update()` changes numbers and asks for a redraw, and GLUT does the rest.

main(), line by line

```
int main(int argc, char **argv)
{
    glutInit(&argc, argv);                // wake GLUT up
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB); // two buffers, colour in R,G,B
    glutInitWindowSize(1920, 1200);        // window size in pixels
    glutInitWindowPosition(40, 40);        // top-left corner on the desktop
    glutCreateWindow("Dinajpur Railway Station");

    glutDisplayFunc(display);              // "when you need the picture, call display"
    initGL();                             // our own one-line setup
    glutSpecialFunc(SpecialInput);         // "arrow keys go to SpecialInput"
    glutKeyboardFunc(NormalInput);         // "letter keys go to NormalInput"
    glutTimerFunc(30, update, 0);          // "call update once, 30 ms from now"

    PlaySound("train_station.wav", NULL, SND_ASYNC | SND_FILENAME | SND_LOOP);

    glutMainLoop();                       // hand control to GLUT and never come back
    return 0;
}
```

main(), lines 3540 – 3569.

Registering a callback is what the three `glut...Func` lines do. You are not calling `display`, `SpecialInput` or `NormalInput`. You are handing GLUT the address of each function and saying "use this one when the time comes". This is why you will never find a line in the file that says `display()`;

Call	What it means
<code>glutInit</code>	starts GLUT and lets it read command-line options
<code>GLUT_DOUBLE</code>	double buffering: draw into a hidden page, then show it all at once
<code>GLUT_RGB</code>	colours are given as red, green and blue
<code>glutCreateWindow</code>	opens the window; the string is the title bar text
<code>glutDisplayFunc</code>	registers the function that draws
<code>glutSpecialFunc</code>	registers the function for arrow / function keys
<code>glutKeyboardFunc</code>	registers the function for ordinary letter and number keys
<code>glutTimerFunc</code>	asks GLUT to call a function once, after a delay in milliseconds
<code>glutMainLoop</code>	the endless loop inside GLUT that waits for events

Why double buffering matters

With a single buffer the user would watch the scene being painted piece by piece and the picture would flicker. With `GLUT_DOUBLE` there are two pages: OpenGL paints the hidden one while the finished one stays on screen, and `glutSwapBuffers()` at the end of `display()` swaps them in one instant. That is the last line of `display()`, line 3535.

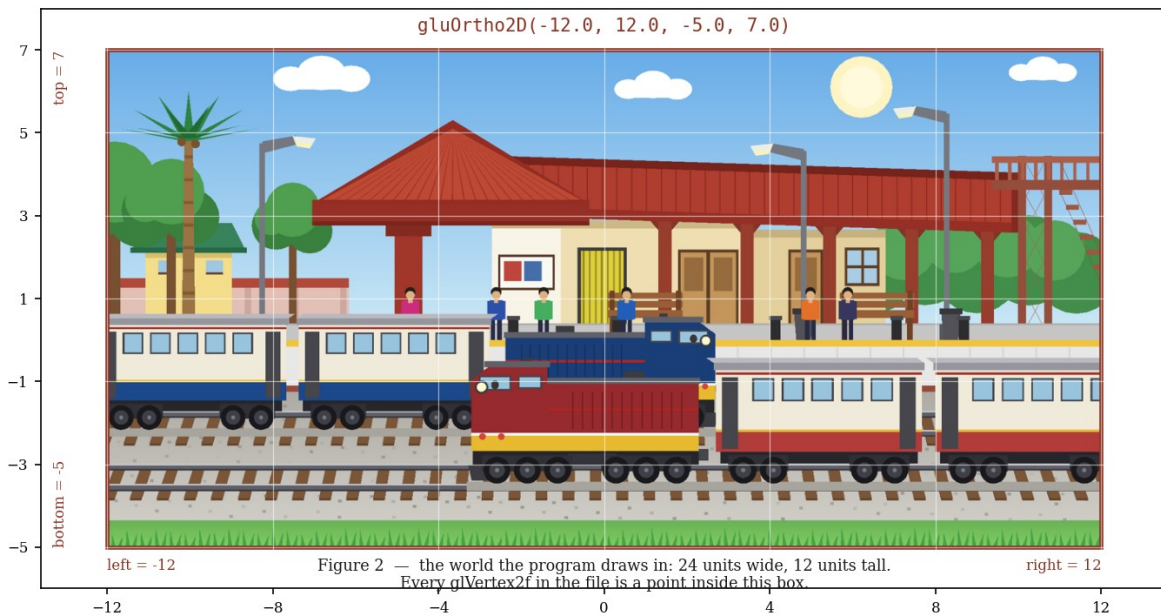
5. The world the drawing lives in

Screen pixels are awkward to draw with, so at the top of `display()` the program sets up its own measuring system:

```
glLoadIdentity();           // throw away any leftover transform
gluOrtho2D(-12.0, 12.0, -5.0, 7.0); // left, right, bottom, top
```

display(), lines 901 and 903.

From then on, *x* runs from -12 on the left edge to $+12$ on the right, and *y* runs from -5 at the bottom to $+7$ at the top. The window is 1920×1200 pixels, so one world unit is 80 pixels across and 100 pixels up. That is where numbers like `glPointSize(7.0f)` come from: a ballast stone is about 0.09 units wide, and $0.09 \times 80 \approx 7$ pixels.



Ortho means there is no perspective: something far away is not drawn smaller automatically. Where the scene looks like it has depth — the sleepers fanning towards the middle of the track — that was done by choosing the corner points by hand, not by the projection.

6. How a single shape gets drawn

Every shape in the file follows the same three steps: choose a colour, open a block, list the corners, close the block.

```
glColor3ub(104, 172, 232); // pick a colour : red, green, blue, each 0-255
glBegin(GL_QUADS);         // "the points I am about to give you are quads"
glVertex2f(-12.0f, 7.0f);  // corner 1
glVertex2f( 12.0f, 7.0f);  // corner 2
glVertex2f( 12.0f, 0.0f);  // corner 3
glVertex2f(-12.0f, 0.0f);  // corner 4
glEnd();                   // done, draw it
```

The sky, display() line 913.

The colour is a state, not a property of the shape. Once you set it, every shape after it uses that colour until you set a different one. This catches people out: if you delete a shape but leave its `glColor3ub` behind, the next shape quietly changes colour.

The five kinds of block this program uses

Primitive	Points per shape	Used for	Blocks
GL_QUADS	4	walls, roofs, windows, sleepers, almost everything	126
GL_TRIANGLES	3	grass blades, the gable ribs	7

Primitive	Points per shape	Used for	Blocks
GL_TRIANGLE_FAN	centre + rim	every round shape, through drawCircle	3
GL_LINES	2	rain, roof corrugations, tile joints, louvre slats	6
GL_POINTS	1	the 460 ballast stones and the 80 stars	6

GL_POINTS and GL_LINES have a thickness set in pixels by glPointSize() and glLineWidth(). Both are reset to 1.0 straight after use, so they do not leak into the next shape.

What glVertex2f actually means

The name is not random. gl is the library, Vertex is the job, 2 is how many numbers you are giving (x and y), and f is the type of those numbers (float). The same pattern explains glColor3ub: three numbers, each an unsigned byte, which is why the range is 0 to 255.

7. Transformations : moving, sizing and the matrix stack

Writing out a wheel at every place a wheel appears would be madness. Instead the program writes one shape at the origin and then moves the pen before drawing it. Four calls do all of this.

Call	Plain meaning
glTranslatef(x, y, 0)	shift everything drawn after this by x and y
glScalef(sx, sy, 1)	stretch everything drawn after this; a minus sign mirrors it
glPushMatrix()	save the current position so you can get back to it
glPopMatrix()	go back to the saved position

Push and pop must always come in pairs. They are a stack — a pile of saved positions. If you translate without pushing first, the shift adds on to the next shift, and the next, and within a second the whole scene has slid off the screen. In this file there are 14 pushes and 14 pops, and they balance. Try commenting out one pair and running it: the scene explodes, and that one experiment will teach you more than a page of explanation.

drawCircle : one ring of points, ninety-two shapes

Every round thing in the scene — 24 wheel tyres, hubs and axle caps, the sun, the moon, nine cloud puffs, nine distant trees, the leafy tree crowns, the coconuts, the lamp glows, the bollard caps — is the same ring of 24 points, stretched to size.

```
void drawCircle(float cx, float cy, float rx, float ry)
{
    glPushMatrix();                // remember where we were
    glTranslatef(cx, cy, 0.0f);    // move the pen to the centre
    glScalef(rx, ry, 1.0f);        // rx and ry set the two radii
    glBegin(GL_TRIANGLE_FAN);
    glVertex2f(0.0f, 0.0f);        // the middle of the fan
    glVertex2f(1.0f, 0.0f);        // then 24 points around a circle of radius 1
    glVertex2f(0.9659f, 0.2588f);
    /* ... 22 more ... */
    glEnd();
    glPopMatrix();                // put the pen back
}
```

```
}
```

drawCircle, lines 256 – 290.

Because *rx* and *ry* are separate, the same function draws a circle when they are equal and an oval when they are not. Twenty-one of the round shapes in the scene are actually ovals — the cloud puffs and the lamp glows — and they come out of this one function.

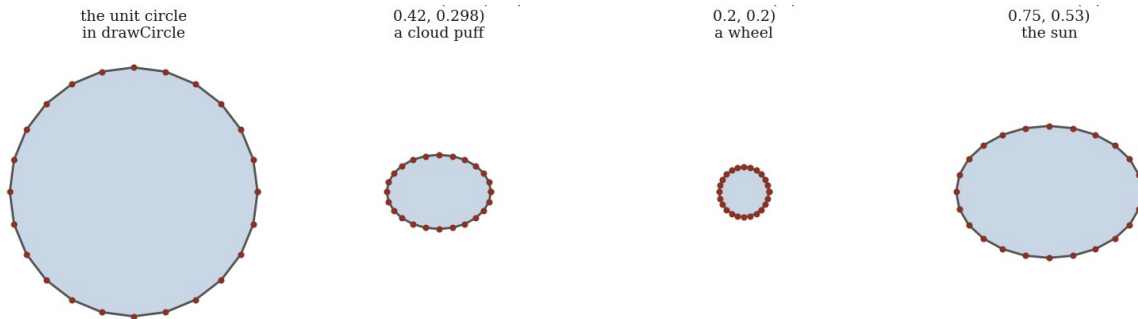


Figure 4 — one ring of 24 points is written down once. *glScalef* stretches it into every round shape in the scene.

GL_TRIANGLE_FAN, and why a circle is made of triangles

A graphics card can only fill triangles. A fan is the cheapest way to describe a disc: give the centre point first, then walk around the rim, and the card fills the wedge between the centre and each neighbouring pair of rim points. Twenty-four wedges look perfectly round at the sizes used here. Change the 24 points to 5 and every wheel, cloud and the sun become pentagons — another one-line experiment worth running once.

8. The repeated objects

The same idea as *drawCircle* is used for five bigger things. In each case the shape is written down once and then stamped wherever it is needed.

Function	Written once	Used	Saved by
<i>drawCircle</i>	a 24-point ring	92 times	<i>glTranslatef</i> + <i>glScalef</i>
<i>drawClouds</i>	three clouds	twice, one world apart	<i>glTranslatef</i>
<i>drawCoach</i>	one coach	3 times per train, 6 in all	<i>glTranslatef</i>
<i>drawTrain</i>	coaches + engine	twice, the second mirrored	<i>glTranslatef</i> + <i>glScalef</i>
<i>drawPassenger</i>	one figure	5 times, different shirt colours	an <i>x</i> parameter
<i>drawBench</i>	one bench	twice	an <i>x</i> parameter

The train

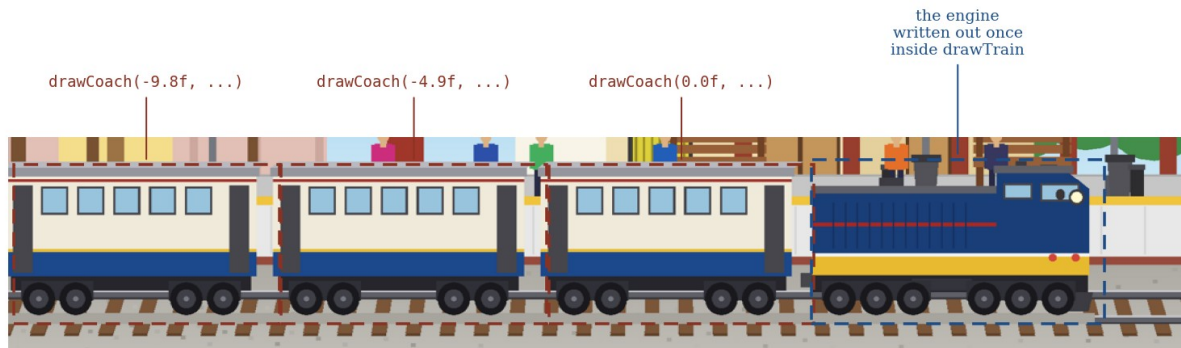


Figure 5 — one coach is written down once, then drawn three times at three offsets. The whole train moves because display() wraps it in glTranslatef(trainPos, 0.0f, 0.0f).

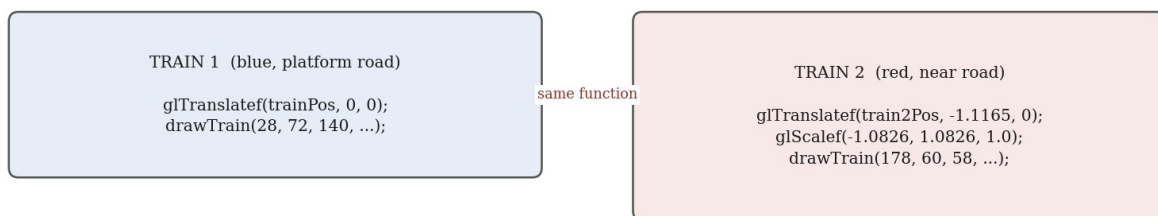
```
/* ---- three coaches, the same one drawn three times ---- */
drawCoach(0.0f, r1, g1, b1); /* the one coupled to the engine */
drawCoach(-4.9f, r1, g1, b1); /* 4.9 is one coach plus the gap */
drawCoach(-9.8f, r1, g1, b1);
```

Inside drawTrain, lines 511 – 514.

A coach is 4.86 units long and the coupling gap is about 0.04, so the spacing is 4.9. Change that single number and the coaches slide apart or overlap — it is the easiest thing in the file to demonstrate.

The second train is the first one in a mirror

The two trains are not two drawings. They are the same drawTrain call twice, and the second one is wrapped in a scale with a minus sign.



$$x' = -1.0826 x \quad y' = 1.0826 y - 1.1165$$

the minus sign flips it left-to-right, 1.0826 makes it slightly bigger,
-1.1165 drops it onto the nearer track

Figure 6 — the second train is the first one seen in a mirror.

```
/* the platform train, in Bangladesh Railway blue */
glPushMatrix();
glTranslatef(trainPos, 0.0f, 0.0f);
drawTrain(28, 72, 140, 26, 62, 120, 20, 48, 96);
glPopMatrix();

/* the same train, mirrored and a little larger, on the near road */
```

```

glPushMatrix();
glTranslatef(train2Pos, 0.0f, 0.0f);
glPushMatrix();
glTranslatef(0.0f, -1.1165f, 0.0f);
glScalef(-1.0826f, 1.0826f, 1.0f); /* the minus turns it to face the other way */
drawTrain(178, 60, 58, 150, 42, 46, 108, 28, 32);
glPopMatrix();
glPopMatrix();

```

display(), the two train calls.

The nine numbers passed to drawTrain are three colours: the body, the darker band and the darkest shade. The -1.0826 flips the train left-to-right and makes it 8 per cent larger; the -1.1165 drops it onto the nearer track. A negative scale also reverses the order the corners are listed in, which would matter if the program used face culling — it does not, so the picture is unaffected. Expect that question.

9. The order things are drawn

There is no depth buffer and no sorting. display() is one long list from the back of the scene to the front, and whatever is drawn later simply covers what came before, exactly like painting on a canvas. This is called the painter's algorithm.



Figure 3 — display() is one long list. Whatever is drawn later covers whatever was drawn earlier, like painting on a canvas.

Two consequences worth knowing. First, the clouds are drawn after the sun and the moon, so they pass in front of them. Second, the rain is the very last thing drawn, so it falls in front of everything including the trains.

10. How the animation works

This is the heart of the project, and it is simpler than it looks. There is no animation library. There is one function, update(), that GLUT calls every 30 milliseconds, and all it does is add small amounts to a handful of numbers. display() then draws the scene using whatever those numbers currently are.

The tick

```
void update(int value)
{
    /* ... change all the moving numbers ... */

    glutPostRedisplay();          // "the picture is out of date, please redraw"
    glutTimerFunc(30, update, 0); // "call me again in another 30 ms"
}
```

The first and last lines of update(), lines 55 and 211 – 212.

The last line is the trick that keeps the whole thing alive. `glutTimerFunc` only fires once, so `update()` books itself again at the end of every visit. `main()` starts the very first tick; after that `update()` keeps itself going forever. 30 milliseconds gives about 33 frames a second.

`glutPostRedisplay()` does not draw anything. It sets a flag that says the window is out of date. GLUT notices the flag when it is next idle and calls `display()` itself. That is why `update()` and `display()` never call each other.

update() is called every 30 milliseconds. It changes numbers, then asks for a redraw.			
trainPos	+= 0.16 each tick	the blue train rolls right	L57-60
train2Pos	-= 0.20 each tick	the red train rolls left	L63-72
sceneClock	+1 each tick	counts to 260, then the hour turns	L76-85
lightR/G/B	chase wantR/G/B by 0.006	the light fades, never jumps	L107-119
nightMix	+/- 0.008	stars, moon and lamp glow fade in	L115-129
sunMix	+/- 0.008	the sun sinks behind the station	L131-144
walk1..walk6	+= walkStep, bounce at limits	the six passengers pace	L146-181
rainDrop	-= 0.28, resets at -2.0	the rain falls	L183-187
cloudDrift	+= 0.012, resets at 24.0	the clouds drift across	L194-197

Figure 7 — everything that moves is one number changing a little bit, thirty-three times a second.

Each moving thing, one at a time

The platform train. One variable holds where it is and another holds how fast.

```
if (trainPos > 27.0f)    /* it has left on the right side */
    trainPos = -20.0f;  /* put it back off the left edge */
trainPos += trainSpeed; /* trainSpeed is 0 when stopped */
```

update(), lines 58 – 60.

`display()` then wraps the whole train in `glTranslatef(trainPos, 0, 0)`, so changing this one number slides the entire train, coaches and all. The DOWN arrow sets `trainSpeed` to 0 and the train stops in place; UP sets it back to 0.16. The threshold is 27 and not 12 because the train is now about 20 units long — it has to leave the screen completely before it is teleported back, or the jump would be visible.

The through train. It sits off the right edge doing nothing until the RIGHT arrow sets `train2Running` to 1. Then it moves left by 0.20 a tick until it is past -29, when it is parked again and the flag cleared.

The six passengers. Each has a position and a step. When the position passes a limit the step is negated, so the figure turns round and walks back.

```

if (walk1 > 2.40f)
    walkStep1 = -0.020f; /* far enough right, turn round */
if (walk1 < -2.60f)
    walkStep1 = 0.020f; /* far enough left, turn round */
walk1 += walkStep1;

```

update(), lines 148 – 152. The other five are identical with different limits.

The rain. One number slides all 280 streaks downwards and snaps back to the top, so the rain appears endless.

```

rainDrop -= 0.28f;
if (rainDrop < -2.0f)
    rainDrop = 0.0f;

```

update(), lines 189 – 191. The streaks are spaced 2.0 apart, so the reset is invisible.

The clouds. Same idea sideways, but with one extra trick. The clouds are drawn twice, one world-width apart, so as the first set leaves the right edge the copy is already coming in from the left.

```

cloudDrift = cloudDrift + 0.012f;
if (cloudDrift > 24.0f)
    cloudDrift = 0.0f;

/* and in display() : */
drawClouds(cloudDrift);
drawClouds(cloudDrift - 24.0f); /* 24 is the width of the world */

```

update() lines 194 – 197, display() lines 1047 – 1048.

The day and night cycle

A counter called sceneClock adds one every tick. When it passes 260 — about 7.8 seconds — the hour turns.

```

if (rainOn == 0) /* the rain holds the clock still */
{
    sceneClock = sceneClock + 1;
    if (sceneClock > 260)
    {
        sceneClock = 0;
        scenePhase = scenePhase + 1; /* 0 = day, 1 = night */
        if (scenePhase > 1)
            scenePhase = 0;
        startAmbience = 1; /* the hour has turned */
    }
}

```

update(), lines 73 – 85.

Wrapping the whole block in `if (rainOn == 0)` is what makes the rain freeze time. The counter does not merely skip the change — it stops counting, so when the rain is turned off the cycle carries on from exactly where it paused.

11. How the light changes colour

Nothing in the scene is redrawn differently at night. There is no second set of night-time shapes. The entire change is one colour, fed to OpenGL's lighting model, which then multiplies every lit shape in the picture.

```

glEnable(GL_LIGHTING); // turn the lighting model on
glEnable(GL_COLOR_MATERIAL); // let glColor3ub act as the material colour

GLfloat global_ambient[] = {lightR, lightG, lightB, 1.0f};

```

```
glLightModelfv(GL_LIGHT_MODEL_AMBIENT, global_ambient);
```

display(), lines 905 – 910. These five lines tint the whole scene.

GL_COLOR_MATERIAL is the piece that makes this work. Normally a lit object needs a material defined separately from its colour. With colour material enabled, the plain glColor3ub you already wrote is treated as the material, so the ambient light multiplies it directly.

update() decides what that colour should be. Each state has a target, and the current colour creeps towards the target rather than jumping to it.

```
if (scenePhase == 0)          /* clear day */
{   wantR = 1.00f; wantG = 1.00f; wantB = 1.00f; }
if (scenePhase == 1)          /* night */
{   wantR = 0.26f; wantG = 0.31f; wantB = 0.58f; }
if (rainOn == 1 && scenePhase == 0) /* rain greys the daylight down */
{   wantR = 0.62f; wantG = 0.68f; wantB = 0.78f; }

/* the light does not jump, it fades slowly to the new colour */
if (lightR < wantR) lightR += 0.006f;
if (lightR > wantR) lightR -= 0.006f;
/* the same two lines for G and B */
```

update(), lines 87 – 119.

Going from white (1.00) to the night blue (0.58) at 0.006 a tick takes about 70 ticks, a little over two seconds. That gentle creep is why dusk looks like dusk instead of a light switch.

nightMix and sunMix

Two more numbers handle the things that appear and disappear rather than change colour. nightMix climbs from 0 to 1 at night and controls the stars, the moon and the glow around the street lamps — each of those is drawn inside an if (nightMix > 0.0f). sunMix does the opposite job for the sun: when it falls, the sun is slid downwards behind the station by glTranslatef, which is a much more reliable way of removing it than fading its colour.

```
if (sunMix > 0.0f)
{
    glPushMatrix();
    glTranslatef(0.0f, -5.0f * (1.0f - sunMix), 0.0f); /* sink the sun */
    /* ... draw the sun ... */
    glPopMatrix();
}
```

display(), the sun.

12. How the sound works

The sound does not come from OpenGL or GLUT. It comes from Windows, through one function in windows.h.

```
PlaySound("rainny.wav", NULL, SND_ASYNC | SND_FILENAME | SND_LOOP);
```

A typical call.

Piece	Meaning
"rainny.wav"	the file name, looked for in the folder the program runs from
NULL	no module handle needed, because we are naming a file
SND_FILENAME	"the first argument is a file name, not a resource name"

Piece	Meaning
SND_ASYNC	start it and return at once; do not freeze the program
SND_LOOP	keep repeating until something else is played

Only one sound can play at a time through this call, and starting a new one stops the old one. That single fact is what the whole sound design has to work around.

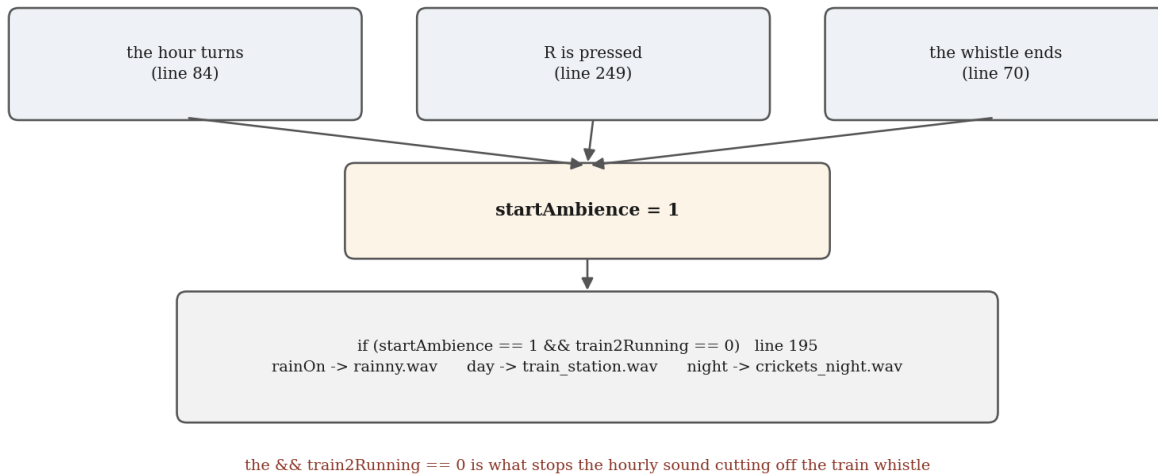


Figure 8 — one flag decides when the looping background sound is swapped.

```

if (startAmbience == 1 && train2Running == 0)
{
    startAmbience = 0;
    if (rainOn == 1)          /* the rain is on, whatever the hour */
        PlaySound("rainny.wav", NULL, SND_ASYNC | SND_FILENAME | SND_LOOP);
    if (rainOn == 0 && scenePhase == 0) /* clear day */
        PlaySound("train_station.wav", NULL, SND_ASYNC | SND_FILENAME | SND_LOOP);
    if (rainOn == 0 && scenePhase == 1) /* night */
        PlaySound("crickets_night.wav", NULL, SND_ASYNC | SND_FILENAME | SND_LOOP);
}

```

update(), lines 195 – 205.

Why the flag? If the program called PlaySound every tick the sound would restart 33 times a second and you would hear nothing but a click. So three places raise a flag — the hour turning, the R key, and the whistle finishing — and this block acts on the flag once and immediately lowers it.

Why the second condition? The RIGHT arrow plays the whistle without SND_LOOP, as a one-shot. If the hour happened to turn while the whistle was sounding, the background loop would cut it off mid-blow. Checking train2Running == 0 holds the background sound back until the through train has finished crossing, and the flag raised at that moment puts it back.

One practical note: the file names have no folder in front of them, so the .wav files must sit in the same folder the program is run from. Running the .exe from a different working directory gives a silent scene.

13. How the keyboard works

GLUT splits the keyboard into two. Ordinary printable keys arrive as a character; arrow keys, function keys and page keys are not characters at all, so they arrive through a different callback as a code. That is why this program has two key functions rather than one.

```
void SpecialInput(int key, int x, int y) /* arrows : registered with glutSpecialFunc */
{
    switch (key)
    {
    case GLUT_KEY_DOWN:
        trainSpeed = 0.0f; /* stop the train at the platform */
        break;
    case GLUT_KEY_UP:
        trainSpeed = 0.16f; /* let the train go again */
        break;
    case GLUT_KEY_RIGHT:
        if (train2Running == 0)
        {
            train2Pos = 20.0f;
            train2Running = 1;
            PlaySound("train_whistle.wav", NULL, SND_ASYNC | SND_FILENAME);
        }
        break;
    }
    glutPostRedisplay();
}
```

SpecialInput, lines 215 – 238.

```
void NormalInput(unsigned char key, int x, int y) /* letters : glutKeyboardFunc */
{
    if (key == 'r' || key == 'R')
    {
        if (rainOn == 0)
            rainOn = 1; /* let it rain */
        else
            rainOn = 0; /* and stop it again */
            startAmbience = 1; /* swap the sound over */
    }
    glutPostRedisplay();
}
```

NormalInput, lines 241 – 252.

Key	Effect	What it changes
UP arrow	the platform train moves	trainSpeed = 0.16
DOWN arrow	the platform train stops where it is	trainSpeed = 0.0
RIGHT arrow	sends the through train across, with a whistle	train2Running = 1
R	rain on / rain off, and freezes the clock	rainOn flips, startAmbience = 1

Note what these functions do not do: they never draw anything. They set a variable and ask for a redraw. Both checking for lower case 'r' and upper case 'R' means the key works whether or not Caps Lock is on.

14. Where to find the code for any job

Use this table in the viva. If you are asked "show me where X happens", this is the answer.

If you are asked about...	Go to
the window, its size and title	main, lines 3544 – 3550
the frame rate / the timer	main line 3560, and update line 212
the coordinate system	display, line 903
the ambient light / day and night colour	update 87 – 119, display 905 – 910
the day-night cycle and its speed	update, lines 73 – 85
the sun rising and setting	update 131 – 144, display 995
stars, moon and lamp glow	update 115 – 129, display 923 and 2255
the rain falling	update 189 – 191, display 3500
the clouds drifting	update 194 – 197, drawClouds 293, display 1047
the platform train moving	update 58 – 60, display 2966
the through train and the whistle	update 63 – 72, SpecialInput 226
the coaches	drawCoach 343, called at 511 – 514
the mirrored second train	display, the glScalef call
the walking passengers	update 146 – 181, drawPassenger 804
every round shape	drawCircle, lines 256 – 290
the sleepers on the track	display 2938 and 3006, the for loops
the sound	update 195 – 205, main 3564
the keyboard	SpecialInput 215, NormalInput 241

15. Every keyword and call used in the file

OpenGL

Call	What it does	Where
glClearColor	sets the colour the window is wiped to	initGL
glClear(GL_COLOR_BUFFER_BIT)	wipes the colour buffer before drawing	display, first line
glLoadIdentity	resets the current matrix to "no transform"	display
gluOrtho2D	sets the 2D coordinate range	display
glBegin / glEnd	opens and closes a block of points	148 times each
glVertex2f	one corner point, x and y as floats	2190 times
glColor3ub	sets the current colour, 0 – 255 per channel	202 times
glPushMatrix / glPopMatrix	save and restore the current position	14 each
glTranslatef	shift everything drawn afterwards	14 times

Call	What it does	Where
glScalef	stretch or mirror everything drawn afterwards	2 times
glPointSize	thickness of GL_POINTS, in pixels	11 times
glLineWidth	thickness of GL_LINES, in pixels	6 times
glEnable / glDisable	turns a feature such as lighting on or off	4 / 3
glLightModelfv	hands OpenGL the ambient light colour	display

GLUT

Call	What it does
glutInit	starts GLUT
glutInitDisplayMode	asks for double buffering and RGB colour
glutInitWindowSize / Position	size and position of the window
glutCreateWindow	opens the window
glutDisplayFunc	registers the drawing function
glutSpecialFunc	registers the arrow-key function
glutKeyboardFunc	registers the letter-key function
glutTimerFunc	call a function once after N milliseconds
glutPostRedisplay	marks the window as needing a redraw
glutSwapBuffers	shows the hidden page; the last line of display
glutMainLoop	the endless event loop; never returns

C++ and Windows

Keyword or call	What it does here
int / float / GLfloat	whole numbers and decimals; GLfloat is OpenGL's own float
GLubyte	a number from 0 to 255, used for colour parameters
void	"this function gives nothing back"
if / else	choose between two paths
for	repeat a block a fixed number of times (sleepers, rain)
while	repeat until a condition fails
switch / case / break	pick one of several paths by value, used for the arrow keys
&& == !=	and, or, is-equal, is-not-equal
+= -=	add to / subtract from a variable in place
a global variable	declared outside every function, so update and display share it
PlaySound	Windows call that plays a .wav file
SND_ASYNC / SND_FILENAME / SND_LOOP	flags joined with to control how it plays

16. Questions you should expect, and the answers

1. Why is there no image file in the project?

Because nothing is an image. Every shape is a list of corner points drawn with `glVertex2f`, so the whole scene is built from geometry.

2. What makes the picture move?

`update()` is called every 30 ms by `glutTimerFunc`. It changes a handful of numbers and calls `glutPostRedisplay`. `display()` redraws the scene from the new numbers.

3. Why does `update()` call `glutTimerFunc` at its own end?

`glutTimerFunc` only fires once. Re-registering at the end of every visit is what keeps the animation going.

4. What is double buffering and why use it?

Two pages of pixels. OpenGL paints the hidden one and `glutSwapBuffers` shows it in one step, so the user never sees a half-drawn frame and there is no flicker.

5. Why must `glPushMatrix` and `glPopMatrix` be paired?

They are a stack of saved positions. Without the pop, each `translate` adds to the last and the scene drifts away within a second.

6. What is the difference between `glutKeyboardFunc` and `glutSpecialFunc`?

Printable keys arrive as a character through `glutKeyboardFunc`. Arrow and function keys are not characters, so they come as codes through `glutSpecialFunc`.

7. How is the second train drawn without writing it again?

The same `drawTrain` function, wrapped in `glScalef(-1.0826, 1.0826, 1.0)`. The minus mirrors it, the 1.0826 enlarges it, and a `translate` drops it onto the nearer track.

8. Does the negative scale cause a problem?

It reverses the winding order of the corners. It would matter if face culling were switched on; the program never calls `glCullFace`, so it makes no difference here.

9. How does the whole scene turn blue at night?

One ambient colour, `lightR/lightG/lightB`, is handed to `glLightModelfv`. With `GL_COLOR_MATERIAL` enabled, it multiplies the `glColor3ub` of every lit shape in the picture.

10. Why does the light fade instead of jumping?

`update` moves the current colour towards the target by 0.006 per tick rather than assigning it, which takes roughly two seconds.

11. Why does the R key freeze the day and night cycle?

The whole clock block sits inside `if (rainOn == 0)`, so while it rains `sceneClock` stops counting. It resumes from the same value when the rain is turned off.

12. How do you stop a sound cutting another one off?

One flag, `startAmbience`, plus the condition `train2Running == 0`, so the looping background sound waits until the whistle and the through train have finished.

13. Why is the sleeper spacing written as a loop?

The 47 sleepers on each track sit on a constant step of 0.6255, so a for loop reproduces them exactly with four vertex calls instead of 188.

14. Why is there no perspective?

gluOrtho2D is an orthographic projection. The depth you can see in the track was drawn in by hand, not produced by the projection.

15. What is immediate mode?

Points are handed to OpenGL every frame and forgotten afterwards. Nothing is stored on the graphics card between frames.

The six questions your teacher has already put to other groups

16. Why use GLUT here?

OpenGL can only draw. It has no idea what a window is, it cannot read a keyboard, and it cannot keep time. Those three jobs belong to the operating system, and they are written completely differently on Windows, Linux and Mac. GLUT is a thin layer that does them for you, the same way on every machine.

In this project GLUT does exactly four jobs, and it is worth naming them: it opens the window (glutCreateWindow, line 3546), it hands us the keys (glutSpecialFunc and glutKeyboardFunc, lines 3550 and 3552), it runs the clock that drives the animation (glutTimerFunc, line 3554), and it shows each finished frame (glutSwapBuffers, the last line of display).

Without GLUT the program would need several hundred lines of raw Windows API code — register a window class, create the window, build a message loop, set up an OpenGL rendering context — before a single line could be drawn. We use freeglut, the free modern version of the original GLUT.

17. What is OpenGL?

OpenGL is a standard set of drawing commands that graphics cards understand. It is not a programming language and not an application: it is an interface, and the actual implementation lives inside your graphics driver. Every card from every maker supports the same commands, which is why the same code runs on any machine.

You hand it three things — a list of corner points, a colour, and a few settings such as whether lighting is on — and it fills in the pixels. It knows nothing about trains, roofs or stations; it only knows points, lines, triangles and quads.

This project uses immediate mode, the oldest and simplest style of OpenGL. Points are handed over one at a time inside glBegin and glEnd, every single frame, and OpenGL forgets them the moment the frame is finished. Nothing is stored on the card between frames.

18. How does the display change?

The display never changes in place. Once a shape has been drawn, it is only pixels — there is no list of objects sitting there that the program can reach back into and move. So to change anything at all, the program wipes the window and draws the whole scene again from the beginning.

That redraw happens many times a second. Every time it runs, the drawing is built out of a small set of variables that describe the state of the scene at that moment. Between one redraw and the next, something changes a few of those variables by a small amount, so each new picture is slightly different from the one before it, and the eye reads the sequence as movement.

The cycle is always the same four steps: clear the window, draw everything, show the finished picture, change a few values. Then it starts again.

Two rules follow from this. Anything that is meant to move or change has to live in a variable, because the picture is rebuilt from those variables every single time. And the change has to be small, because a large jump in a value is a visible jump on screen.

19. What are the features?

A day and night cycle that runs on its own, turning every 260 ticks, about 7.8 seconds, with the light fading gradually between the two rather than snapping.

Rain on the R key. Pressing R starts the rain and freezes the clock where it stands, so the hour stops turning until R is pressed again.

Two trains, each an engine and three coaches. The platform train is driven by the UP and DOWN arrows; the through train crosses once per press of the RIGHT arrow, with a whistle. The second train is the first one mirrored with a negative scale.

Six passengers walking the platform, each with its own speed and its own two turning points, so no two ever walk in step.

A sky that works: the sun sinks behind the station as night comes on, while the moon, eighty stars and the glow around three street lamps fade up in its place, and clouds drift across and wrap round to the other side.

Sound: three looping ambiances, one for day, one for night and one for the rain, plus a one-shot whistle that the looping sound politely waits out.

20. What is Ortho2D?

`gluOrtho2D(left, right, bottom, top)` sets the coordinate system you draw in. Ours is `gluOrtho2D(-12.0, 12.0, -5.0, 7.0)`, so *x* runs from -12 at the left edge to +12 at the right, and *y* from -5 at the bottom to +7 at the top. Every `glVertex2f` in the file is a point inside that box, and OpenGL stretches the box to fit the window.

The word *ortho* means orthographic, which is a parallel projection: there is no vanishing point and no perspective. A shape is exactly the same size whether you place it at the front of the scene or the back. That is the opposite of `gluPerspective`, which is the 3D call where distant things shrink.

This matters for one question you may be asked. The two tracks in this scene look as if they recede into the distance, but that depth was drawn in by hand — the sleeper coordinates were chosen to fan inwards. The projection contributes nothing to it.

`gluOrtho2D` is simply the convenience form of `glOrtho` for flat work: it is the same call with the near and far planes fixed at -1 and +1.

21. What did you use for moving objects?

`glTranslatef`, driven by variables that `update()` changes on every tick. That is the whole answer; everything else is detail.

A shape is written down once at fixed coordinates. Before it is drawn, `glTranslatef` shifts the origin by the current value of a variable, so the shape appears somewhere else. Change the variable a little on every tick and the shape moves.

The moving parts and their variables: `trainPos` for the platform train, `train2Pos` for the through train, `walk1` to `walk6` for the six passengers, `cloudDrift` for the clouds, `rainDrop` for the rain, and `sunMix` for the sun sinking behind the station.

Two supporting pieces make it work. `glutTimerFunc(30, update, 0)` is the clock — and because a GLUT timer fires only once, `update()` re-books itself at its own last line. `glPushMatrix` and `glPopMatrix` wrap every move, so one object's shift does not carry over to the next; without them the scene slides off the screen within a second.

`glScalef` is used as well, but not for movement: it sizes the unit circle inside `drawCircle`, and it mirrors the second train with a negative *x* factor. `glRotatef` is not used anywhere in this project.

17. Building and running it

```
g++ main2_reduced.cpp -o station.exe -lfreeglut -lopengl32 -lglu32 -lwinmm -lgdi32
```

One line, from the folder that holds the .cpp and the five .wav files.

Library	Why it is needed
-lfreeglut	GLUT itself: the window, the keyboard, the timer
-lopengl32	OpenGL: every gl... call
-lglu32	the GLU helper, for gluOrtho2D
-lwinmm	Windows multimedia, for PlaySound
-lgdi32	Windows graphics, which freeglut needs underneath

Run the .exe from the folder that holds the .wav files, or the scene will be silent. In Code::Blocks, check that the project's working directory is the project folder and not the compiler's bin folder — that is a common cause of a silent build.